Statistical Dependency Is Not Logical Dependency: Logical Equivariance for Diffusion Language Model Reasoning

Alice[1*] **Bob**[1*] **Tom**[1] **Bard**[1] **Alex**[1]
[1]UofT
{Alice, Bob}@gmail.com

Abstract

Diffusion language models (dLLMs) can denoise masked sequences in arbitrary order, suggesting an advantage for reasoning tasks whose dependencies are not left-to-right. We show that arbitrary order is not enough: a decoder can be statistically dependency-aware while violating task-level logical equivariance. We define logical equivariance as consistency under transformations that preserve a problem's dependency graph, and introduce ANCHORED COMPLETION, a diagnostic that clamps all logical evidence while changing only the 1D serialization geometry. On 104 anchored Sudoku instances, only 15.4% are solved correctly across all tested transformations, while 83.7% exhibit layout-dependent success or failure. Comparing model-derived dependency graphs with ground-truth logical graphs, we find that model dependencies can track sequence geometry rather than logical structure. Denoising traces further reveal false-anchor cascades, where early unsupported commitments distort later predictions. Our results suggest that dLLM reasoning requires generation order to follow logical dependency, not merely model confidence or statistical dependency.

1 Introduction

Diffusion language models (dLLMs) have recently emerged as a serious alternative to autoregressive language models. Instead of generating tokens strictly from left to right, masked diffusion language models iteratively denoise partially masked sequences, allowing bidirectional context use, infilling, parallel token updates, and arbitrary-order generation [**?** **?**]. This flexibility appears especially attractive for reasoning. In a proof, the next useful premise need not be the next sentence. In a math problem, the relevant quantities may be scattered across the prompt. In a constraint satisfaction problem, the next variable to resolve may be the most constrained variable, not the next variable in the serialized string. If generation order is free, dLLMs seem to offer a natural route beyond the left-to-right bottleneck of autoregressive decoding.

However, arbitrary order is not logical order. Recent dLLM decoding methods primarily address a model-level sampling problem: how to unmask multiple tokens in parallel without deviating too far from the model's own conditional distribution. Fast-dLLM identifies quality degradation in parallel decoding as a consequence of disrupting token dependencies under a conditional-independence approximation, and mitigates this with confidence-aware decoding [**?**]. DAPD constructs an attention-induced dependency graph over masked tokens and selects weakly coupled positions for parallel unmasking [**?**]. DEMASK estimates pairwise conditional influence and selects positions whose cumulative model dependency is bounded [**?**]. These methods improve the speed–quality or accuracy–steps trade-off of dLLM sampling. Yet they optimize fidelity to the model distribution,

[*]Equal contribution

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

not necessarily fidelity to the logical structure of the task. If the model's own dependency structure is biased by the 1D serialization of the input, then a decoder can be statistically dependency-aware while still being logically wrong.

This paper studies that gap. We ask whether dLLMs and their decoders preserve the logical symmetries of the reasoning problems they solve. When two inputs differ only by a transformation that preserves the underlying dependency graph, a reasoning model should produce correspondingly transformed answers. We call this property *logical equivariance*. For an input transformation T and its corresponding output action S_T, a model or decoder f should satisfy

$$f(Tx) \approx S_T f(x).$$

In Sudoku, rotating or reflecting the board changes the 1D text order of cells but preserves the constraint graph; the completed grid should rotate or reflect in the same way. In graph coloring, relabeling nodes should relabel the coloring but not change whether the assignment is valid. In a math word problem, reordering independent premises or renaming variables should preserve the answer or transform it in a predictable way. Violations of this principle reveal a form of brittleness that average accuracy can hide: a model may solve one presentation of a problem while failing on an equivalent one.

This failure mode is not unique to Sudoku. Many reasoning tasks can be viewed as computation over latent dependency graphs presented through 1D text. Sudoku makes this graph explicit; math word problems hide it behind entity–quantity bindings; proofs expose it as a premise-to-conclusion DAG; and code exposes it as data-flow and control-flow dependencies. Prior work has already documented symptoms of this broader problem. Language models can fail to robustly use relevant information when its position changes in a long context [**?**]. Premise order can substantially affect deductive and mathematical reasoning even when the underlying task is unchanged [**?**]. GSM-Symbolic shows that controlled symbolic perturbations, additional clauses, and no-op information can substantially degrade mathematical reasoning performance [**?**]. These works show that strong benchmark accuracy does not imply invariance to dependency-preserving transformations. What remains unclear is why such failures occur, whether dLLMs avoid them through arbitrary-order generation, and what training signals are required for dLLMs to acquire such invariance.

We use Sudoku and related constraint satisfaction problems (CSPs) as controlled microscopes for this question. This choice is deliberate. Natural-language benchmarks such as GSM8K and MATH mix arithmetic, language understanding, world knowledge, discourse structure, prompt format, and decoding behavior. When a model fails on such examples, it is difficult to isolate whether the cause is calculation error, linguistic ambiguity, irrelevant context, or position bias. CSPs remove these confounds. Their variables, constraints, dependency graphs, and symmetry groups are explicit. Transformations such as rotations, reflections, symbol permutations, node relabelings, and valid row/column permutations preserve the logical problem exactly. Thus, Sudoku is not the application of our work; it is an identifiability device. It lets us observe the pathology under exact transformations before testing the same principle in noisier math, proof, and code settings.

To expose the pathology, we introduce ANCHORED COMPLETION. In this diagnostic, all logical evidence required for a completion state is clamped in the input, while the model fills only a specified set of masked target variables. We then apply dependency-preserving transformations that keep the logical state fixed but change the 1D serialization geometry. Under logical equivariance, solving a transformed instance and mapping the answer back to the canonical coordinate system should recover the same solution. Instead, we find severe orbit inconsistency. On 9×9 Sudoku AC16 solver-window completion, LLaDA-8B achieves nontrivial per-transformation accuracy: 50.3%, 48.8%, and 50.0% on easy, medium, and hard splits respectively. However, only 1.6%, 1.5%, and 2.3% of puzzles are solved correctly under all eight D_4 transformations, while more than 95% are layout-sensitive. In aggregate, LLaDA solves 97.7% of base puzzles under at least one transformed presentation, but only 1.8% under all eight. Thus, the model is not simply unable to solve the instances; rather, the same logical state can be solved or broken depending on its layout.

This observation motivates our central thesis: *statistical dependency is not logical dependency*. A dLLM decoder may estimate which tokens are strongly coupled under the model distribution, yet those couplings may align with sequence proximity rather than with the task's logical graph. We therefore compare model-derived dependency graphs against ground-truth logical graphs. The model graph G_{model} can be obtained from attention, decoder-specific dependency scores, or sampled influence estimates. The task graph G_{logic} is known exactly in CSPs. We ask whether model dependency scores

correlate with logical distance d_{logic} or with serialized text distance d_{seq}. This comparison directly tests whether dependency-aware decoding recovers task structure or merely follows 1D positional biases.

We further exploit a feature unique to diffusion language models: their denoising trajectories are observable. Unlike autoregressive models, which commit to a left-to-right generation order by design, dLLMs expose when each variable is unmasked, committed, remasked, or revised. We trace finalization order and compare it to text order, solver-derived logical order, and constraint difficulty. This reveals a failure pattern we call a *false-anchor cascade*: an early high-confidence but logically unsupported commitment becomes a spurious anchor, after which subsequent denoising steps adapt around the wrong value rather than correcting it. This mechanism explains why confidence-aware or model-dependency-aware decoding can still fail under logical equivariance tests.

Diagnosis alone is not enough. We also ask what it takes for dLLMs to *grok* Sudoku in the sense of logical equivariance. Prior work on logic puzzles shows that Transformer models can learn to solve Sudoku when trained on solver-derived logical sequences, suggesting that the training signal matters as much as the architecture [**?**]. However, simply imitating a single solver-order sequence is not sufficient for our question. We study small masked diffusion language models at multiple scales and introduce LEAP, a logical equivariance anchored pre-training and post-training recipe. LEAP converts solver traces into solver-window denoising states, aligns the denoising distribution across symmetry orbits, encourages near-solution fixed-point stability, and optionally distills recovery trajectories from false-anchor failures. The goal is not merely to increase Sudoku accuracy, but to make orbit consistency, model–logical dependency alignment, and false-anchor stability emerge during training.

Finally, we test whether the same principle extends beyond Sudoku. We introduce TEMPLATE-ANCHORED BINDING COMPLETION for natural-language math, where a reasoning skeleton is clamped and only typed semantic slots such as entities, quantities, intermediate values, and final answers are masked. This turns premise reordering from a coarse answer-level robustness test into a slot-level variable-binding test. We also construct Proof-DAG Anchored Completion and Code Anchored Completion, where dependency-preserving transformations correspond to premise reordering, variable renaming, alpha-renaming, and independent statement reordering. These bridge tasks do not provide the same causal identifiability as exact CSPs. Their purpose is to test whether the equivariance principle exposed by Sudoku also appears in math, proof, and code settings.

Our contributions are:

1. We operationalize **logical equivariance** as a task-level criterion for diffusion language model reasoning under dependency-preserving transformations.
2. We introduce **ANCHORED COMPLETION**, an exact CSP diagnostic that clamps logical evidence while varying only serialization geometry. On 9×9 Sudoku AC16, LLaDA-8B solves 97.7% of puzzles under at least one transformed presentation but only 1.8% under all eight D_4 transformations.
3. We show that **statistical dependency is not logical dependency** by comparing model-derived dependency graphs with ground-truth logical graphs and measuring whether dependency scores align with d_{logic} or d_{seq}.
4. We perform a **denoising trajectory autopsy** and identify false-anchor cascades, where early high-confidence but logically unsupported commitments distort later predictions.
5. We study **what it takes for small dLLMs to grok Sudoku** by scaling masked diffusion models at fixed puzzle size and comparing standard training, solver-window diffusion, orbit-consistent training, and LEAP.
6. We connect exact CSP failures to broader reasoning through **TABC, Proof-AC, and Code-AC**, which evaluate variable binding, proof-dependency, and data-flow equivariance under dependency-preserving transformations.

Together, these results suggest that the promise of dLLMs for reasoning is not merely their ability to generate in arbitrary order. The central challenge is to make generation order, dependency estimation, and denoising dynamics respect the logical dependency graph of the task. Without such alignment, a dLLM may escape left-to-right decoding while remaining trapped by a deeper 1D positional bias.

2 Related Work

Diffusion language models. Diffusion models for language have developed along both continuous and discrete directions. Diffusion-LM introduced a continuous diffusion formulation over word embeddings for controllable text generation [**?**]. D3PMs extended diffusion models to discrete state spaces with structured corruption processes, including absorbing-state transitions that connect diffusion to mask-based generative modeling [**?**]. More recently, masked diffusion language models (MDLMs) showed that a simple masked discrete diffusion objective can be competitive with autoregressive language modeling when paired with modern training recipes [**?**]. Large Language Diffusion Models such as LLaDA scale masked diffusion to the LLM regime by learning a reverse masked-token prediction process [**?**]. Dream 7B further demonstrates that discrete diffusion LMs can support parallel iterative denoising, arbitrary-order generation, infilling, and tunable quality–speed trade-offs [**?**].

These works establish dLLMs as a viable alternative to autoregressive LMs. However, they do not directly answer whether arbitrary-order generation corresponds to logically valid generation order. Our work studies this missing question: whether the denoising distribution and trajectory of a dLLM preserve the task-level dependency structure of reasoning problems.

Parallel and dependency-aware decoding for dLLMs. A major bottleneck of dLLMs is inference efficiency. Since masked diffusion models may require many denoising steps, recent work has explored ways to unmask multiple tokens in parallel. Fast-dLLM introduces approximate KV caching and confidence-aware parallel decoding, identifying quality degradation as a consequence of disrupting token dependencies under a conditional-independence approximation [**?**]. DAWN similarly accelerates dLLM inference by extracting token dependencies and avoiding simultaneous updates to strongly coupled uncertain positions [**?**]. DAPD constructs a conditional dependency graph from self-attention and selects an independent set of weakly coupled tokens for parallel unmasking [**?**]. DEMASK estimates pairwise conditional influence between masked positions and selects positions with bounded cumulative dependency, providing a distributional bound on the gap between parallel sampling and the model's joint conditional [**?**].

Our work is closely related to these methods but asks a different question. Existing dependency-aware decoders aim to remain faithful to the model's own distribution: they reduce the mismatch between parallel token updates and the model-implied joint conditional. We instead ask whether the model's dependency structure is faithful to the task's logical dependency graph. A decoder can be statistically dependency-aware with respect to p_θ while still violating task-level logical equivariance. In this sense, our evaluation is complementary to speed–quality analyses: we test whether dependency-aware decoding preserves the logical symmetries of the problem.

Generation order and reasoning in masked diffusion models. Several recent works show that token ordering is central to the reasoning behavior of masked diffusion models. **?**] argue that MDMs face difficult infilling subproblems at training time but can exploit inference-time flexibility by choosing favorable token decoding orders; on Sudoku, adaptive inference can substantially improve pretrained MDM accuracy. **?**] learn order policies for masked diffusion models and evaluate on token-order-sensitive tasks such as Sudoku and 3-SAT. **?**] propose Order-Token Search, which jointly searches over generation order and token values for diffusion LM reasoning. The d1 framework adapts pretrained masked dLLMs into reasoning models using masked supervised fine-tuning and diffu-GRPO reinforcement learning, evaluating on mathematical, planning, and logical reasoning benchmarks [**?**].

These works motivate our focus on generation order, but our target is different. We do not primarily seek a better search algorithm or faster sampler. Instead, we ask whether a generation order is logically valid. Our claim is that "arbitrary order" and even "adaptive order" are insufficient criteria for reasoning unless the resulting trajectory is aligned with the task's logical dependency graph. This distinction motivates our comparison between G_{model} and G_{logic}.

Solver-order learning and logic-puzzle reasoning. Logic puzzles provide a useful testbed for studying whether sequence models can acquire search and reasoning behavior. **?**] show that Transformer causal language models can learn to solve Sudoku and Zebra puzzles when trained on logical sequences of solver steps. In Sudoku, their model achieves high full-puzzle correctness

when trained on solver-derived logical traces, while models trained without such logical sequence supervision fail to learn the task. This result is important for our work because it shows that the training signal, not only model capacity, determines whether a Transformer can internalize the structure of Sudoku.

Our work differs in two ways. First, we study masked diffusion language models rather than causal language models. Second, we do not train a model to imitate a single canonical solver-order sequence. Instead, we convert solver traces into solver-window Anchored Completion states and train the denoising distribution to be consistent over the symmetry orbit of the same logical state. In short, prior solver-order training teaches a path to the solution; our LEAP training studies how dLLMs can learn a solution manifold that is equivariant under dependency-preserving transformations.

Reasoning, fixed points, and grokking. Recent mechanistic analyses suggest that reasoning models may appear to solve puzzles while relying on brittle or spurious internal dynamics. Work on reasoning-versus-guessing in HRM-style models reports fixed-point violations, sudden grokking-like transitions, and multiple attractor states in Sudoku-like settings [**?**]. These observations resonate with our false-anchor analysis: a model may commit to an early high-confidence but unsupported value and then rationalize subsequent predictions around that commitment.

Our contribution is to study this phenomenon in diffusion language models, where the denoising trajectory exposes when variables are unmasked, committed, remasked, or revised. This lets us measure false-anchor cascades directly and train against them through false-anchor recovery distillation and fixed-point stability losses.

Order sensitivity and brittle reasoning in LLMs. A growing body of work shows that LLM reasoning is sensitive to surface form and input order. **?**] show that language models can fail to robustly use relevant information when its position in a long context changes. **?**] demonstrate that premise ordering can substantially affect deductive and mathematical reasoning even when the underlying task is unchanged, and introduce R-GSM to study ordering effects in mathematical word problems. GSM-Symbolic constructs symbolic variants of GSM8K problems and shows that model performance can vary across different instantiations of the same template; it further shows that adding no-op information can substantially degrade performance despite not contributing to the required reasoning chain [**?**].

We view these phenomena as symptoms of a broader failure: models are not reliably invariant or equivariant to transformations that preserve the underlying dependency structure. Prior work documents the brittleness; our work provides an exact dependency-graph microscope for isolating it. Sudoku and related CSPs allow us to control the logical graph, the transformation group, and the output action exactly, thereby separating logical position bias from linguistic ambiguity, arithmetic skill, or world knowledge.

Semantic invariance, logical invariance, and metamorphic testing. Semantic invariance and metamorphic testing have recently been proposed as reliability criteria for LLM reasoning. Metamorphic testing evaluates systems by applying transformations that should preserve expected behavior, such as paraphrase, fact reordering, expansion, contraction, or contextual reframing [**?**]. Semantic-invariance frameworks evaluate whether LLM agents remain stable under meaning-preserving input variations across reasoning domains [**?**]. Related work on logical invariance uses symmetry pairs and contrastive consistency tuning to reduce contradictory outputs under logically equivalent prompts [**?**].

Our work is narrower but more mechanistic. Rather than testing broad semantic invariance under natural-language transformations, we focus on logical equivariance under dependency-preserving transformations. In exact CSPs, the transformation group, dependency graph, and corresponding output action are known. In natural language, premise reordering, variable renaming, and no-op insertion are treated as noisier external-validity tests of the same principle. This distinction lets us move beyond response stability and ask whether model-internal dependency aligns with the task-level logical graph.

Equivariance, symmetry, and structured reasoning. Equivariance has a long history in machine learning as a way to encode known symmetries into model behavior. Group-equivariant convolutional networks exploit transformations such as rotations and reflections to improve generalization [**?**].

In graph learning, Transformer-based models such as Graphormer show that structural information must be explicitly encoded for attention mechanisms to work well over graph-structured data [**?**]. These lines of work highlight a general principle: models benefit when their architecture, training, or evaluation respects the geometry of the problem.

Our setting differs from standard equivariant architectures and graph Transformers in two ways. First, we study language models and their decoding or post-training behavior rather than building a task-specific equivariant architecture from scratch. Second, our main concern is not simply whether a model can represent graph structure, but whether the dependency structure induced by its attention, logits, or denoising decisions aligns with the task's logical dependency graph. This motivates our comparison between G_{model} and G_{logic}.

Position encodings and sequence geometry. Transformers process reasoning problems through 1D serialized token sequences. Relative and rotary position encodings make sequence position available to attention and have been central to modern LLMs [**? ?**]. These position mechanisms are useful in language modeling, where textual proximity often correlates with syntactic or discourse-level dependency. However, in structured reasoning tasks, logical proximity need not correspond to textual proximity. A cell in the same Sudoku column, a distant premise in a proof, or an earlier independent fact in a word problem may be logically adjacent while being far apart in the serialized prompt.

We do not claim that any single positional encoding is solely responsible for reasoning failure. Instead, we study a broader mismatch between 1D sequence geometry and task-level logical geometry. Logical equivariance provides a way to measure this mismatch directly: if changing only serialization geometry changes the model's answer, then the model has failed to preserve the logical structure of the task.

Bridging CSPs to math, proof, and code reasoning. A common concern with Sudoku is that it may appear disconnected from general reasoning. We treat Sudoku not as the application, but as a microscope: it exposes the dependency graph, transformations, and output action exactly. To connect this exact setting to broader reasoning domains, we introduce bridge tasks. In Template-Anchored Binding Completion, math word problems are converted into typed entity, quantity, intermediate-value, and answer slots, allowing us to test variable-role binding under premise reordering or no-op insertion. In Proof-DAG Anchored Completion, proof facts and intermediate conclusions form an explicit dependency DAG. In Code Anchored Completion, variable bindings and data-flow edges define the logical graph, and transformations include alpha-renaming and independent statement reordering.

These bridge tasks are intentionally more controlled than full free-form math or code generation. Their purpose is not to claim that Sudoku training alone solves general reasoning, but to test whether the same dependency-preserving equivariance principle appears in arithmetic binding, proof dependencies, and code data flow.

Our position relative to prior work. The closest prior works improve dLLM decoding by estimating model-level token dependencies, searching over token orders, or post-training dLLMs for reasoning. Other related work evaluates LLM robustness under premise reordering, semantic transformations, or symbolic perturbations of math word problems. Our work introduces a different axis of evaluation and training. We ask whether the model and decoder are equivariant to dependency-preserving transformations of the task.

This leads to five differences. First, we evaluate entire transformation orbits rather than single canonical prompts. Second, we compare model-derived dependency graphs against ground-truth logical dependency graphs. Third, we use Sudoku and CSPs as exact microscopes, then test external validity on math, proof, and code bridge tasks. Fourth, we analyze dLLM denoising trajectories to identify false-anchor cascades. Fifth, we study how logical equivariance can emerge through scale and LEAP post-training, rather than merely asking whether a pretrained model succeeds or fails.

In short, prior dLLM decoding work asks how to decode efficiently and faithfully with respect to the model distribution. Prior robustness work asks whether LLM outputs are stable under semantic transformations. We ask whether the distribution, the decoder, and the training dynamics are faithful to the logical symmetries of the reasoning problem.

3 Preliminaries

4 Methodology

4.1 Overview

This section describes how we evaluate and train diffusion language models (dLLMs) for logical equivariance. Our methodology is built around three questions. First, do pretrained dLLMs preserve logical equivariance under *dependency-preserving transformations*? Second, what scale and training signal are required for a small dLLM to grok Sudoku under logical equivariance? Third, can we use the same principle as post-training for larger dLLMs and for bridge tasks in math, proof, and code?

The central distinction is between *model-level statistical dependency* and *task-level logical dependency*. Existing dependency-aware dLLM decoders aim to unmask multiple tokens while remaining faithful to the model's own distribution [**? ? ?**]. We instead ask whether the model distribution, decoder decisions, and training dynamics respect the dependency graph of the task.

Our methodology has six components:

1. We define logical equivariance for reasoning instances represented by dependency graphs.
2. We introduce , an exact diagnostic that clamps all logical evidence while changing only presentation geometry.
3. We define orbit-level metrics that evaluate consistency across equivalent presentations rather than only average accuracy.
4. We compare model-derived dependency graphs against task-level logical graphs.
5. We analyze dLLM denoising trajectories to identify false-anchor cascades.
6. We introduce , a pre-training and post-training recipe that trains orbit-consistent denoising, fixed-point stability, and false-anchor recovery.

4.2 Reasoning Instances and Logical Equivariance

Let x denote a reasoning instance and $y^\star(x)$ its correct solution. We associate each instance with a task-level logical dependency graph

$$G_{\text{logic}}(x) = (V, E_{\text{logic}}),$$

where vertices correspond to logical units such as variables, cells, facts, clauses, entities, code variables, or intermediate reasoning states. An edge $(i, j) \in E_{\text{logic}}$ indicates that the value or validity of i directly depends on j.

A serialized prompt induces a 1D position map

$$\sigma_x : V \to \{1, \dots, n\}.$$

For $i, j \in V$, define sequence and logical distances as

$$d_{\text{seq}}(i, j) = |\sigma_x(i) - \sigma_x(j)|, \qquad d_{\text{logic}}(i, j) = \text{dist}_{G_{\text{logic}}(x)}(i, j).$$

The mismatch between these geometries is the object of study. In structured reasoning, two logical units may be adjacent in the task graph while far apart in the serialized text.

Dependency-preserving transformations. Let $\mathcal{T}$ be a set of transformations. A transformation $T \in \mathcal{T}$ is *dependency-preserving* if it changes the presentation of the instance while preserving the dependency graph up to a vertex permutation π_T:

$$(i, j) \in E_{\text{logic}}(x) \iff (\pi_T(i), \pi_T(j)) \in E_{\text{logic}}(Tx).$$

The output transforms under a corresponding action S_T:

$$y^\star(Tx) = S_T y^\star(x).$$

For an exact CSP, T may rotate, reflect, relabel, or permute the structured state. For natural language, T may reorder independent premises or rename variables. For code, T may alpha-rename variables or reorder independent statements.

Logical equivariance. A deterministic decoder f is logically equivariant with respect to $\mathcal{T}$ if

$$f(Tx) = S_T f(x) \qquad \forall T \in \mathcal{T}.$$

For a probabilistic model, the corresponding distributional condition is

$$p_\theta(y \mid x) = p_\theta(S_T y \mid Tx).$$

In experiments, we evaluate decoder-level equivariance. Given decoder $\mathcal{D}_\theta$ and seed r, we map each transformed prediction back to the canonical frame:

$$\tilde{y}_{T,r} = S_T^{-1} \mathcal{D}_\theta(Tx; r).$$

A logically equivariant decoder should produce the same canonical answer across the entire transformation orbit.

4.3 Why Model Fidelity Is Insufficient

The distinction between statistical dependency and logical dependency is formal.

Proposition 4.1 (Model fidelity does not imply logical equivariance)**.** *Let $\mathcal{T}$ be a set of dependency-preserving transformations with output action S_T. Suppose p_θ is not logically equivariant; that is, there exists $T \in \mathcal{T}$ and y such that*

$$p_\theta(y \mid x) \neq p_\theta(S_T y \mid Tx).$$

Then any decoder that faithfully samples from, or minimizes distributional discrepancy to, $p_\theta(\cdot \mid x)$ can still violate logical equivariance.

Proof sketch. If p_θ assigns different probabilities to equivalent outputs across a dependency-preserving transformation, then a decoder faithful to p_θ inherits this asymmetry. Reducing joint–marginal mismatch with respect to p_θ can make the decoder more faithful to the model distribution, but cannot correct non-equivariance already present in that distribution. Thus, model-level sampling fidelity is insufficient unless either p_θ itself is task-equivariant or the training/decoding procedure explicitly enforces task-level structure. □

This proposition separates two questions that are often conflated: whether a decoder respects the model distribution, and whether the model distribution respects the task symmetry.

4.4 for Exact Dependency Graphs

is a controlled diagnostic for isolating logical position bias. For a reasoning instance x, let $A(x) \subset V$ be anchored variables whose values are fixed in the input, let $U(x) \subset V$ be target variables to complete, and let $F(x) = V \setminus (A(x) \cup U(x))$ be future unknown variables not evaluated in the current instance. During dLLM denoising, anchors are clamped:

$$z_t(i) = y_i^\star \qquad \forall i \in A(x),\ \forall t.$$

Targets are initialized as `[MASK]` and may be denoised. Future variables are represented by and are excluded from the loss. The model is evaluated only on $U(x)$. This removes shortcut explanations based on missing evidence: all logical evidence needed for the current completion state is present and fixed.

Solver-window Anchored Completion. For CSPs with solver traces, let

$$s_1, s_2, \ldots, s_m$$

be a deterministic sequence of logical assignments. Given window length k and position t, define

$$A_t = \text{givens} \cup \{s_1, \ldots, s_{t-1}\},$$

$$U_{t,k} = \{s_t, \ldots, s_{t+k-1}\}, \qquad F_{t,k} = \{s_{t+k}, \ldots, s_m\}.$$

The model receives A_t, masks $U_{t,k}$, and treats $F_{t,k}$ as future unknowns. AC16 and AC32 denote $k = 16$ and $k = 32$. This converts a solver trace into a family of logical states rather than a single output sequence.

Exact transformations. For square-grid CSPs such as Sudoku, we use the dihedral group

$$\mathcal{T}_{D_4} = \{\mathrm{id}, \mathrm{rot}_{90}, \mathrm{rot}_{180}, \mathrm{rot}_{270}, \mathrm{flip}_h, \mathrm{flip}_v, \mathrm{diag}, \text{anti-diag}\}.$$

We also consider symbol permutations and valid row/column permutations when they preserve the constraint graph. For graph coloring, transformations include node relabeling, adjacency-list permutation, and color-symbol permutation. For proof and SAT-style tasks, transformations include variable renaming, clause-order permutation, and dependency-preserving topological reordering.

Serialization. Unless otherwise specified, every transformed grid instance is serialized in row-major order. Applying T and then serializing row-major changes which logical dependencies are close in 1D text while preserving the underlying task graph. Additional serializations are reported as ablations.

4.5 Bridge Tasks Beyond Exact CSPs

Exact CSPs provide causal isolation, but math, proof, and code are the broader target settings. We therefore define controlled bridge tasks that expose analogous variable-binding and dependency-preserving transformations.

Template-Anchored Binding Completion. Given a math word problem x, we obtain a reasoning skeleton $c(x)$ from a gold symbolic template or a correct model-generated solution. We replace entities, quantities, intermediate values, and the final answer with typed slots:

$$c(x) \to c_{\mathrm{mask}}(x; \mathcal{S}),$$

where $\mathcal{S}$ is the set of semantic slots. For a transformation T, such as premise reordering, name renaming, or no-op insertion, the model is conditioned on Tx and the clamped skeleton $c_{\mathrm{mask}}(x; \mathcal{S})$, and fills only the slots. This tests whether the model binds values to logical roles rather than to their surface order.

Proof-DAG Anchored Completion. For synthetic proof tasks, vertices are premises, intermediate facts, or conclusions, and edges are premise-to-conclusion dependencies. Transformations include premise reordering, variable renaming, irrelevant premise insertion, and valid topological reordering. The model fills masked proof-step slots or final entailment labels.

Code Anchored Completion. For code, the logical graph is a data-flow or control-flow graph. We construct small Python-style programs with independent assignment reordering, alpha-renaming, and no-op statement insertion. The model fills masked values, variables, or outputs. Correctness is evaluated by slot accuracy and unit-test equivalence.

These bridge tasks do not provide the same causal identifiability as exact CSP symmetries. Their purpose is to test whether the same dependency-preserving equivariance principle appears in natural-language and code settings.

4.6 Orbit-Level Evaluation Metrics

Average accuracy can hide failures of logical equivariance. We therefore evaluate performance over entire transformation orbits.

Per-transformation accuracy. For base instance x, transformation T, and seed r,

$$\mathrm{Acc}(x, T, r) = \mathbb{I}\big[S_T^{-1}\mathcal{D}_\theta(Tx; r) = y^\star(x)\big] .$$

Orbit accuracy.

$$\mathrm{OrbitAcc}(x) = \frac{1}{|\mathcal{T}||\mathcal{R}|} \sum_{T\in\mathcal{T}} \sum_{r\in\mathcal{R}} \mathrm{Acc}(x, T, r).$$

Correct orbit consistency.

$$\mathrm{CorrectOrbitCons}(x) = \mathbb{I}\big[\forall T \in \mathcal{T},\ S_T^{-1}\mathcal{D}_\theta(Tx) = y^\star(x)\big] .$$

Orbit consistency and symmetry violation. Orbit consistency ignores correctness and asks whether canonicalized predictions are identical across the orbit:

$$\mathrm{OrbitCons}(x) = \mathbb{I}\big[\forall T, T' \in \mathcal{T},\ S_T^{-1}\mathcal{D}_\theta(Tx) = S_{T'}^{-1}\mathcal{D}_\theta(T'x)\big]\,.$$

The symmetry violation rate is

$$\mathrm{SVR} = 1-$$

rac1|X| $\sum_{x\in\mathcal{X}} \mathrm{OrbitCons}(x)$.

Serialization sensitivity and worst-case orbit drop. Let

$$\bar{a}_{x,T} = \frac{1}{|\mathcal{R}|}\sum_{r\in\mathcal{R}} \mathrm{Acc}(x,T,r), \qquad \bar{a}_x = \frac{1}{|\mathcal{T}|}\sum_{T\in\mathcal{T}} \bar{a}_{x,T}.$$

Then

$$\mathrm{SSI}(x) = \sqrt{\frac{1}{|\mathcal{T}|}\sum_{T\in\mathcal{T}} (\bar{a}_{x,T} - \bar{a}_x)^2},$$

and

$$\mathrm{WOD}(x) = \max_{T\in\mathcal{T}} \bar{a}_{x,T} - \min_{T\in\mathcal{T}} \bar{a}_{x,T}.$$

Slot-level bridge metrics. For TABC, Proof-AC, and Code-AC, each slot $s \in \mathcal{S}$ has gold value $v_s^\star$. We report

$$\mathrm{SlotAcc}(x,T) = \frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}} \mathbb{I}[\hat{v}_s(Tx) = v_s^\star].$$

For entity-value slots, the binding swap rate is

$$\mathrm{SwapRate}(x,T) = \mathbb{I}\Big[\exists s_i, s_j : \hat{v}_{s_i} = v_{s_j}^\star \wedge \hat{v}_{s_j} = v_{s_i}^\star\Big]\,.$$

A hidden binding error occurs when the final answer is correct but an intermediate role binding is wrong:

$$\mathrm{HBE}(x,T) = \mathbb{I}[\hat{a} = a^\star \wedge \exists s \in \mathcal{S}_{\mathrm{intermediate}} : \hat{v}_s \neq v_s^\star]\,.$$

4.7 Model Dependency vs. Logical Dependency

Let $s_\theta(i,j)$ be a model-derived dependency score between logical units or token positions i and j. We compare these scores with G_{logic}.

Attention-derived dependency. For standard dLLM decoding, we average attention over selected layers, heads, and denoising steps:

$$s_{\mathrm{attn}}(i,j) = \frac{1}{|\mathcal{L}||\mathcal{H}||\mathcal{T}_d|}\sum_{\ell\in\mathcal{L}}\sum_{h\in\mathcal{H}}\sum_{t\in\mathcal{T}_d} A_{ij}^{(\ell,h,t)}.$$

Decoder-provided dependency. For dependency-aware decoders, we use their native scores when available, such as the attention-induced graph in DAPD or pairwise influence scores in DEMASK.

Sampled intervention-derived dependency. For diagnostic analysis, we estimate influence by clamping or remasking j and measuring the change in the marginal distribution at i:

$$s_{\mathrm{infl}}(i,j) = D_{\mathrm{KL}}\left(q_{\theta,t}(z_i \mid x, m_t) \| q_{\theta,t}(z_i \mid x, m_t^{j\leftarrow\mathrm{clamped}})\right).$$

Computing this exhaustively is expensive, so we estimate it only at critical steps and on a sampled pair set. The primary critical step is the one with highest mean predictive entropy:

$$t^\star = \arg\max_t \frac{1}{|U(x)|}\sum_{i\in U(x)} H(q_{\theta,t}(z_i \mid x)).$$

We optionally include the first false-anchor step when available. At selected steps, we evaluate pairs in

$$\mathcal{P} = \mathcal{P}_{\mathrm{logic}} \cup \mathcal{P}_{\mathrm{seq}} \cup \mathcal{P}_{\mathrm{rand}},$$

where $\mathcal{P}_{\mathrm{logic}}$ contains logical neighbors, $\mathcal{P}_{\mathrm{seq}}$ contains sequence-near non-logical pairs, and $\mathcal{P}_{\mathrm{rand}}$ is a matched random control set.

Alignment metrics. We report edge-prediction AUC:

$$\mathrm{AUC}_{\mathrm{logic}} = \mathrm{AUC}\left(s_\theta(i,j), \mathbb{I}[(i,j) \in E_{\mathrm{logic}}]\right),$$

and correlations with logical and sequence distance:

$$\rho_{\mathrm{logic}} = \mathrm{corr}_{(i,j)\in\mathcal{P}}(s_\theta(i,j), -d_{\mathrm{logic}}(i,j)),$$
$$\rho_{\mathrm{seq}} = \mathrm{corr}_{(i,j)\in\mathcal{P}}(s_\theta(i,j), -d_{\mathrm{seq}}(i,j)).$$

The logical dependency gap is

$$\mathrm{LDG} = \rho_{\mathrm{logic}} - \rho_{\mathrm{seq}}.$$

A negative LDG indicates stronger alignment with 1D sequence proximity than with the logical graph.

4.8 Denoising Trajectory Analysis

dLLMs expose a denoising trajectory, allowing us to analyze how outputs are formed. For position i, define finalization time

$$\tau_i = \min\{t : z_t(i) = z_{t+1}(i) = \cdots = z_T(i),\ z_T(i) \neq \texttt{[MASK]}\}.$$

We compare finalization order against text order, solver-derived logical order, and difficulty order:

$$\tau_{\mathrm{text}} = \mathrm{KendallTau}(\tau, \sigma_x), \qquad \tau_{\mathrm{logic}} = \mathrm{KendallTau}(\tau, \sigma_{\mathrm{logic}}).$$

False-anchor cascades. A false anchor is an incorrect value that is committed early and remains unchanged until the final output. Let

$$t^\dagger = \min\{t : \exists i,\ z_t(i) \neq y_i^\star \text{ and } z_t(i) = z_{t+1}(i) = \cdots = z_T(i)\}.$$

Let $i^\dagger$ be the first false-anchor variable. Its effect on logical neighbors is

$$\mathrm{Cascade}(i^\dagger) = \sum_{j\in\mathcal{N}_{\mathrm{logic}}(i^\dagger)} \left[\log q_{\theta,t^\dagger+1}(y_j^\star \mid x) - \log q_{\theta,t^\dagger-1}(y_j^\star \mid x)\right].$$

We also track constraint violations before and after the false anchor.

4.9 : Training Logical Equivariance

We introduce , a pre-training and post-training recipe for logical equivariance in dLLMs. converts solver traces into anchored denoising states, trains the model distribution to agree across symmetry orbits, encourages fixed-point stability near correct states, and optionally distills recovery from false-anchor failures.

Base target-only masked diffusion loss. For Anchored Completion state (A, U, F), let $M \subseteq U$ be the masked subset of target variables. Anchors are clamped and future variables are represented by . The target-only loss is

$$\mathcal{L}_{\mathrm{AC}} = \mathbb{E}_{M\subseteq U} \sum_{i\in M} -\log p_\theta(y_i^\star \mid z_{\bar{M}}, M, A, F).$$

Orbit-Consistent Solver-Window Diffusion. trains the denoising distribution to agree across the orbit of the same logical state. For $T, T' \in \mathcal{T}$, define canonicalized distributions

$$\bar{p}_T = S_T^{-1} p_\theta(\cdot \mid T x_{\mathrm{AC}}), \qquad \bar{p}_{T'} = S_{T'}^{-1} p_\theta(\cdot \mid T' x_{\mathrm{AC}}).$$

The orbit loss is

$$\mathcal{L}_{\mathrm{orbit}} = \mathbb{E}_{T,T'} D_{\mathrm{KL}}(\bar{p}_T \| \bar{p}_{T'}).$$

Thus

$$\mathcal{L}_{\text{OC-SWD}} = \mathcal{L}_{\mathrm{AC}} + \lambda_{\mathrm{orbit}} \mathcal{L}_{\mathrm{orbit}}.$$

Unlike solver-order teacher forcing, this objective does not train a single canonical output sequence. It trains a masked denoising distribution to be equivariant over the orbit of a logical state.

Logical Fixed-Point Stability Training. A model that has internalized a logical state should preserve correct or near-correct states under denoising. creates near-solution states by revealing most target variables and masking or corrupting a small subset:

$$\mathcal{L}_{\text{fp}} = \mathbb{E}_{z_{\text{near}}} \sum_{i \in M_{\text{near}}} -\log p_\theta(y_i^\star \mid z_{\text{near}}).$$

This teaches the solution basin, not merely a path to a solution.

False-Anchor Recovery Distillation. uses model failures as training data. Given a failed denoising trajectory, we identify the first false anchor $i^\dagger$, remask it and its logical neighborhood, and use an oracle solver or constraint checker to provide the recovery target:

$$\mathcal{L}_{\text{repair}} = -\log p_\theta \left(y_{i^\dagger}^\star \mid \text{Remask}(z_{t^\dagger}, i^\dagger, \mathcal{N}_{\text{logic}}(i^\dagger)) \right),$$

with optional neighborhood loss

$$\mathcal{L}_{\text{nb}} = \sum_{j \in \mathcal{N}_{\text{logic}}(i^\dagger)} -\log p_\theta(y_j^\star \mid z_{t^\dagger}^{\text{repair}}).$$

Optional dependency-contrastive alignment. For controlled small models, we optionally align model dependency scores with logical edges. Positive pairs are logical edges $(i, j) \in E_{\text{logic}}$; hard negatives are sequence-near non-logical pairs:

$$\mathcal{L}_{\text{dep}} = -\log \frac{\exp(s_\theta(i,j)/\eta)}{\exp(s_\theta(i,j)/\eta) + \sum_{k \in \mathcal{N}_{\text{hard}}(i)} \exp(s_\theta(i,k)/\eta)}.$$

Full objective. The complete objective is

$$\mathcal{L}_{\text{LEAP}} = \mathcal{L}_{\text{AC}} + \lambda_{\text{orbit}}\mathcal{L}_{\text{orbit}} + \lambda_{\text{fp}}\mathcal{L}_{\text{fp}} + \lambda_{\text{repair}}\mathcal{L}_{\text{repair}} + \lambda_{\text{nb}}\mathcal{L}_{\text{nb}} + \lambda_{\text{dep}}\mathcal{L}_{\text{dep}}.$$

In ablations, we add these components incrementally to separate the effects of solver-window states, orbit consistency, fixed-point stability, false-anchor recovery, and dependency alignment.

Small-model pre-training. For controlled scaling studies, we train masked diffusion language models at multiple parameter scales while keeping puzzle size, evaluation distribution, and transformation set fixed. This tests whether scale alone induces logical equivariance or whether scale must be paired with the right training signal.

Large-model post-training. For pretrained dLLMs such as LLaDA or Dream, is applied as masked SFT or LoRA post-training. We distinguish Sudoku-only post-training from mixed logical-equivalence post-training over exact CSPs, TABC, Proof-AC, and Code-AC. This avoids claiming that Sudoku-specific post-training alone solves general reasoning.

4.10 : Bounded Logical Remasking

is a bounded inference-time intervention for tasks with explicit or automatically checkable constraints. It is not test-time augmentation and does not vote over transformed inputs. It operates within a single denoising trajectory.

At step t, a base decoder proposes committed positions C_t and values $\hat{z}_{C_t}$. We form

$$\tilde{z}_t = z_t \oplus \hat{z}_{C_t}.$$

Let $\mathcal{C}(x)$ be the task constraints. The violated constraints are

$$\mathcal{V}(\tilde{z}_t; x) = \{c \in \mathcal{C}(x) : \tilde{z}_t \text{ violates } c\}.$$

If nonempty, we remask variables involved in violated constraints and their logical neighborhood:

$$R_t = \bigcup_{c \in \mathcal{V}(\tilde{z}_t; x)} \mathcal{N}_{\text{logic}}(c).$$

Anchors are never remasked:

$$z_{t+1}(i) = \begin{cases} \texttt{[MASK]}, & i \in R_t \setminus A(x), \\ \tilde{z}_t(i), & \text{otherwise.} \end{cases}$$

To prevent unbounded loops, uses a maximum remasking budget $B_{\max}$ and a cap $R_{\max}$ on remasked variables. If the budget is exhausted, it falls back to the base decoder's current highest-probability assignment.

Algorithm 1

Require: Input x; anchors $A(x)$; constraints $\mathcal{C}(x)$; base decoder $\mathcal{D}_\theta$; horizon T; budgets $B_{\max}, R_{\max}$

Initialize z_0 with anchors clamped, targets masked, and future variables as

$b \leftarrow 0$ **for** $t = 0$ *to* $T-1$ **do**

  $(C_t, \hat{z}_{C_t}) \leftarrow \mathcal{D}_\theta(z_t, x)$

  $\tilde{z}_t \leftarrow z_t \oplus \hat{z}_{C_t}$

  $\mathcal{V}_t \leftarrow \{c \in \mathcal{C}(x) : \tilde{z}_t \text{ violates } c\}$ **if** $\mathcal{V}_t \neq \emptyset$ *and* $b < B_{\max}$ **then**

    $R_t \leftarrow \bigcup_{c \in \mathcal{V}_t} \mathcal{N}_{\text{logic}}(c)$

    keep the top-$R_{\max}$ elements of R_t by violation involvement **forall** $i \in R_t \setminus A(x)$ **do**

      $z_{t+1}(i) \leftarrow$ [MASK]

    forall $i \notin R_t \setminus A(x)$ **do**

      $z_{t+1}(i) \leftarrow \tilde{z}_t(i)$

    $b \leftarrow b + 1$ **else**

    $z_{t+1} \leftarrow \tilde{z}_t$

Return z_T

Scope. applies to explicit-constraint tasks such as Sudoku, Latin squares, graph coloring, SAT, and proof-DAG completion. For open-ended natural language, logical equivariance is primarily an evaluation criterion; online remasking is applied only when a symbolic, arithmetic, proof, or code verifier is available.

4.11 Statistical Protocol

All metrics are computed at the level of base instances, not transformed prompts, so that multiple transformations of the same instance form a paired orbit. We report means with 95% bootstrap confidence intervals over base instances. Decoder comparisons use paired randomization tests over identical instance orbits.

Invalid outputs are counted as incorrect and reported separately. For exact CSPs, we report both exact-solution accuracy and constraint-satisfaction rate. For bridge tasks, slot values and final answers are parsed with the same parser across all models and transformations.

For pre-training and post-training experiments, we report metrics as curves over training steps:

$$\text{Accuracy}, \quad \text{CorrectOrbitCons}, \quad \text{SVR}, \quad \text{LDG}, \quad \text{FalseAnchorRate}, \quad \tau_{\text{logic}}, \quad \tau_{\text{text}}.$$

These curves distinguish ordinary task learning from the emergence of logical equivariance.

5 Experiments

We evaluate whether dLLMs preserve logical equivariance, why they fail when they do not, and what training signals can make logical equivariance emerge. Our experiments are organized around five questions:

1. Do pretrained dLLMs solve equivalent presentations of the same logical state consistently?
2. Are failures explained by misalignment between model-derived dependency and task-level logical dependency?
3. Do denoising trajectories reveal false-anchor cascades?
4. What scale and post-training signals are required for small dLLMs to grok Sudoku under logical equivariance?

Table 1: LLaDA-8B on 9×9 Sudoku Anchored Completion under the full D_4 orbit. "Acc." is average per-transformation accuracy. "Avg. Corr." is the average number of correct transformations out of eight. "All-8" is correct orbit consistency. "Any" is the fraction of base puzzles solved under at least one transformation.

Window	Split	Acc.	Avg. Corr. / 8	All-8	Any	Layout-Sens.
AC16	Easy	50.3	4.02	1.6	97.9	96.3
AC16	Medium	48.8	3.91	1.5	97.5	96.0
AC16	Hard	50.0	4.00	2.3	97.7	95.4
AC16	Avg.	49.7	3.98	1.8	97.7	95.9
AC32	Easy	4.0	0.32	0.0	25.7	25.7
AC32	Medium	4.2	0.33	0.0	26.0	26.0
AC32	Hard	3.5	0.28	0.0	23.0	23.0
AC32	Avg.	3.9	0.31	0.0	24.9	24.9

5. Does the same equivariance principle expose failures in math, proof, and code bridge tasks?

5.1 Experimental Setup

Models. We evaluate LLaDA-8B and Dream-7B as representative diffusion language models, along with an autoregressive baseline of comparable scale. For controlled scaling studies, we train small masked diffusion language models with approximately 5M, 10M, 50M, and 100M parameters.

Tasks. Our exact suite contains 9×9 Sudoku Anchored Completion, 4×4 Sudoku, Latin Square completion, and graph coloring. For external validity, we evaluate TABC on GSM-style and GSM-Symbolic-style math problems, Proof-AC on synthetic entailment DAGs, and Code-AC on Python-style data-flow programs.

Decoders. For pretrained dLLMs, we compare confidence decoding, random unmasking, text-order unmasking, Fast-dLLM, DAPD, DEMASK, and CTLR wrappers where constraints are explicit.

Evaluation. All metrics are computed over base instances, not transformed prompts. Each base instance is evaluated over its transformation orbit. Unless otherwise specified, Sudoku uses the full D_4 orbit of size eight.

5.2 Pretrained dLLMs Lack Logical Equivariance

We first evaluate pretrained LLaDA-8B on 9×9 Sudoku Anchored Completion under the full D_4 orbit. We use solver-window Anchored Completion: all givens and solver prefix cells are clamped, the next k solver steps are masked targets, and future unsolved cells are treated as unknowns. AC16 and AC32 denote $k = 16$ and $k = 32$.

Table 1 reports results across easy, medium, and hard puzzle splits. AC16 is the main diagnostic regime: the model achieves nontrivial per-transformation accuracy but almost never solves the entire orbit consistently. AC32 is a stress regime where most instances fall into a floor region.

The AC16 result reveals the central pathology. LLaDA solves 97.7% of base puzzles under at least one D_4 presentation, yet only 1.8% under all eight presentations. Thus, the model is not simply unable to solve the instances. Instead, most instances have at least one favorable layout and at least one failing layout.

Transformation-level asymmetry. Table 2 reports per-transformation accuracy for AC16 averaged across splits. Some transformed views are substantially more favorable than others, despite all preserving the same logical problem.

Table 2: LLaDA-8B AC16 accuracy by transformation, averaged across easy, medium, and hard splits.

Transformation	Accuracy
identity	44.5
rot90	53.5
rot180	48.5
rot270	52.5
flip-lr	43.7
flip-ud	48.2
transpose	53.6
anti-diagonal	53.2

Table 3: Model comparison on 9×9 Sudoku AC16. Fill in Dream and AR values after running the same D4-orbit protocol.

Model	Acc.	Avg. Corr. / 8	All-8	Any	Layout-Sens.
LLaDA-8B	49.7	3.98	1.8	97.7	95.9
Dream-7B	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
AR baseline	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]

Comparison to Dream and AR baselines. We evaluate Dream-7B and the AR baseline under the same AC16 protocol. The goal is not to rank models by Sudoku accuracy, but to test whether non-autoregressive denoising reduces orbit inconsistency.

5.3 Bridge Tasks Expose the Same Principle Beyond Sudoku

To avoid treating Sudoku as the application, we next evaluate dependency-preserving transformations in math, proof, and code bridge tasks. These tasks are noisier than exact CSPs but test whether the same equivariance principle appears in broader reasoning settings.

Template-Anchored Binding Completion. TABC clamps a reasoning skeleton and masks only typed slots. We use GSM-style and GSM-Symbolic-style templates, focusing on premise reordering, entity renaming, no-op insertion, and non-commutative operations.

Proof-DAG and Code Anchored Completion. For Proof-AC, we evaluate proof-step slot completion under premise reordering and variable renaming. For Code-AC, we evaluate data-flow slot completion under alpha-renaming and independent statement reordering.

5.4 Model Dependency Does Not Necessarily Match Logical Dependency

We next compare model-derived dependency graphs with task-level logical graphs. For Sudoku, G_{logic} connects variables sharing a row, column, or box. For graph coloring, it is the input graph. For proof tasks, it is the premise-to-conclusion DAG.

A key signature of logical position bias is $\text{LDG} < 0$: the model-derived dependency graph aligns more with 1D sequence proximity than with the logical graph.

5.5 Denoising Trajectories Reveal False-Anchor Cascades

We analyze denoising trajectories to understand how equivariance failures arise. For each target variable, we record finalization time, confidence, entropy, and whether it is revised. A false anchor is an incorrect value committed before the final step and never changed afterward.

We further compare finalization order with text order and solver-derived logical order.

Table 4: TABC results. Slot-level metrics reveal failures hidden by answer accuracy. HBE is hidden binding error: the final answer is correct but at least one intermediate slot is bound incorrectly.

Dataset	Transform	Model	Answer Acc.	Slot Acc.	Swap Rate	HBE
GSM-TABC	Premise reorder	LLaDA-8B	[FILL]	[FILL]	[FILL]	[FILL]
GSM-TABC	Premise reorder	Dream-7B	[FILL]	[FILL]	[FILL]	[FILL]
GSM-TABC	Premise reorder	AR baseline	[FILL]	[FILL]	[FILL]	[FILL]
GSM-Symbolic-TABC	No-op insert	LLaDA-8B	[FILL]	[FILL]	[FILL]	[FILL]
GSM-Symbolic-TABC	No-op insert	Dream-7B	[FILL]	[FILL]	[FILL]	[FILL]
Non-commutative TABC	Premise reorder	LLaDA-8B	[FILL]	[FILL]	[FILL]	[FILL]
Non-commutative TABC	Premise reorder	Dream-7B	[FILL]	[FILL]	[FILL]	[FILL]

Table 5: Proof and code bridge tasks. Code Pass is unit-test equivalence. These tasks test whether logical equivariance extends beyond grid CSPs.

Task	Transform	Model	Slot Acc.	Equiv. Violation	Code Pass / Proof Acc.
Proof-AC	Premise reorder	LLaDA-8B	[FILL]	[FILL]	[FILL]
Proof-AC	Variable rename	LLaDA-8B	[FILL]	[FILL]	[FILL]
Proof-AC	Premise reorder	Dream-7B	[FILL]	[FILL]	[FILL]
Code-AC	Alpha rename	LLaDA-8B	[FILL]	[FILL]	[FILL]
Code-AC	Statement reorder	LLaDA-8B	[FILL]	[FILL]	[FILL]
Code-AC	Statement reorder	Dream-7B	[FILL]	[FILL]	[FILL]

5.6 Dependency-Aware Decoding Does Not Guarantee Logical Equivariance

We evaluate whether existing dependency-aware dLLM decoders improve logical equivariance. Table 9 compares confidence decoding, Fast-dLLM, DAPD, and DEMASK. These methods are designed to improve model-level sampling fidelity or speed–quality trade-offs; our metrics test whether they also improve task-level orbit consistency.

5.7 Scaling Small dLLMs: What Does It Take to Grok Sudoku?

We train small masked diffusion language models from scratch to study how scale and training signal influence logical equivariance at a fixed puzzle size. We use 9×9 Sudoku AC16 as the main fixed evaluation distribution and AC32 as a stress distribution. Models have approximately 5M, 10M, 50M, and 100M parameters.

We compare four training recipes: Standard MDLM-AC, Solver-window Diffusion, OC-SWD, and LEAP.

We also plot training curves over checkpoints: accuracy, correct orbit consistency, LDG, false-anchor rate, and finalization-order alignment. These curves distinguish ordinary task learning from the emergence of logical equivariance.

5.8 Post-Training Larger dLLMs with LEAP

We apply LEAP as masked SFT or LoRA post-training to pretrained dLLMs. We distinguish Sudoku-only post-training from mixed logical-equivalence post-training.

Sudoku-only post-training tests whether exact logical equivariance can be induced in a controlled domain. Mixed post-training tests whether the training principle improves analogous binding and equivalence metrics outside Sudoku.

5.9 CTLR as a Bounded Intervention

We evaluate CTLR as a bounded intervention for explicit-constraint tasks. CTLR wraps a base decoder: the base decoder proposes token commitments, and CTLR remasks commitments that create task-level violations along with their logical neighborhood.

Table 6: Alignment between model-derived dependency and ground-truth logical dependency. Negative LDG indicates stronger alignment with sequence proximity than logical proximity.

Task	Model	Score	$\mathrm{AUC}_{\mathrm{logic}}$	ρ_{logic}	ρ_{seq}	LDG
Sudoku AC16	LLaDA-8B	Attention	[FILL]	[FILL]	[FILL]	[FILL]
Sudoku AC16	LLaDA-8B	Clamping-KL	[FILL]	[FILL]	[FILL]	[FILL]
Sudoku AC16	Dream-7B	Attention	[FILL]	[FILL]	[FILL]	[FILL]
TABC	LLaDA-8B	Attention	[FILL]	[FILL]	[FILL]	[FILL]
Code-AC	LLaDA-8B	Attention	[FILL]	[FILL]	[FILL]	[FILL]

Table 7: Denoising trajectory statistics. FA Rate is the fraction of failed trajectories with at least one false anchor. Cascade Size is the average number of logical neighbors whose correct-value probability decreases after the first false anchor.

Task	Model	FA Rate	First FA Step	Cascade Size	Δ Violations
Sudoku AC16	LLaDA-8B	[FILL]	[FILL]	[FILL]	[FILL]
Sudoku AC16	Dream-7B	[FILL]	[FILL]	[FILL]	[FILL]
TABC	LLaDA-8B	[FILL]	[FILL]	[FILL]	[FILL]
Code-AC	LLaDA-8B	[FILL]	[FILL]	[FILL]	[FILL]

5.10 Summary of Findings

Our experiments support four conclusions. First, pretrained dLLMs can solve many logical instances under at least one presentation while failing almost all full transformation orbits. Second, model-derived dependency can align with sequence geometry rather than task-level logical dependency. Third, denoising trajectories expose false-anchor cascades, where early unsupported commitments corrupt later predictions. Fourth, logical equivariance does not emerge from arbitrary-order generation alone; it must be evaluated and, when possible, trained through orbit-consistent and stability-oriented objectives.

The key message is that statistical dependency is not logical dependency. The relevant question for dLLM reasoning is not only whether the model can generate in arbitrary order, but whether its generation order and dependency estimates respect the logical structure of the task.

6 Conclusion

Table 8: Finalization-order alignment. A model that follows logical dependency should align more with logical order than with text order.

Task	Model	τ_{text}	τ_{logic}	$\tau_{\text{difficulty}}$
Sudoku AC16	LLaDA-8B	[FILL]	[FILL]	[FILL]
Sudoku AC16	Dream-7B	[FILL]	[FILL]	[FILL]
TABC	LLaDA-8B	[FILL]	[FILL]	[FILL]
Code-AC	LLaDA-8B	[FILL]	[FILL]	[FILL]

Table 9: Existing dLLM decoders under logical equivariance metrics. Decoders may improve average accuracy or reduce steps while still failing to maintain orbit consistency.

Task	Decoder	Acc.	All-8	SVR	Steps	Wall-time
Sudoku AC16	Confidence	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Sudoku AC16	Fast-dLLM	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Sudoku AC16	DAPD	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Sudoku AC16	DEMASK	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
TABC	Confidence	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
TABC	DAPD	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Code-AC	Confidence	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Code-AC	DAPD	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]

Table 10: Small dLLM scaling on fixed 9×9 Sudoku AC16. All-8 measures correct orbit consistency. LDG measures whether model dependency aligns more with logical distance than sequence distance.

Params	Training	Acc.	All-8	SVR	LDG	FA Rate
5M	Standard MDLM-AC	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
5M	Solver-window	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
5M	OC-SWD	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
5M	LEAP	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
10M	Standard MDLM-AC	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
10M	Solver-window	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
10M	OC-SWD	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
10M	LEAP	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
50M	Standard MDLM-AC	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
50M	Solver-window	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
50M	OC-SWD	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
50M	LEAP	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
100M	Standard MDLM-AC	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
100M	Solver-window	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
100M	OC-SWD	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
100M	LEAP	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]

Table 11: Post-training larger dLLMs. Sudoku-only LEAP tests domain-specific correction. Mixed LEAP tests whether the same logical-equivariance principle improves bridge tasks beyond Sudoku.

Model	Post-training	Sudoku All-8	Sudoku LDG	TABC Slot Acc.	Code Pass	Proof Slot Acc.
LLaDA-8B	none	1.8	[FILL]	[FILL]	[FILL]	[FILL]
LLaDA-8B	Sudoku-only LEAP	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
LLaDA-8B	Mixed LEAP	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Dream-7B	none	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Dream-7B	Sudoku-only LEAP	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Dream-7B	Mixed LEAP	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]

Table 12: CTLR intervention and overhead. CTLR is not optimized for speed; its purpose is to test whether task-level logical rollback reduces false anchors and improves orbit consistency.

Task	Decoder	Acc.	All-8	FA Rate	Steps	Extra Steps	Wall-time
Sudoku AC16	Confidence	[FILL]	[FILL]	[FILL]	[FILL]	–	[FILL]
Sudoku AC16	Confidence+CTLR	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Sudoku AC16	DAPD	[FILL]	[FILL]	[FILL]	[FILL]	–	[FILL]
Sudoku AC16	DAPD+CTLR	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]
Code-AC	Verifier-guided	[FILL]	[FILL]	[FILL]	[FILL]	–	[FILL]
Code-AC	Verifier-guided+CTLR	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]	[FILL]

Acknowledgments

This work was supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2024-00457882, AI Research Hub Project, 10%) and (RS-2022-II220311, Development of Goal-Oriented Reinforcement Learning Techniques for Contact-Rich Robotic Manipulation of Everyday Objects, 90%).

References

NeurIPS Paper Checklist

Table of contents

Orbit self-consistency. We also evaluate an orbit-level self-consistency variant in Appendix **??**. This variant decodes several symmetry-transformed inputs and selects the inverse-mapped candidate with the lowest constraint violation. We treat this as an evaluation-time upper bound rather than our main decoding method, since it is closer to test-time augmentation and self-consistency than to an active denoising intervention.